Tier 3 | From indexes to databases

Keep the workload fixed

Use identical IDs, vectors, metric, filters, query split and exact truth. Explicitly supply embeddings so a database does not silently choose a different model. Separate client memory from server memory.

Pick a deployment boundary

Embedded systems share your process and local storage. Server systems add network, authentication, readiness and operational ownership. An index library still needs an application contract for metadata, deletion and persistence.

Native versus adapted

The common VectorStore contract makes comparison possible. Native lessons separately demonstrate features such as named vectors, hybrid fusion, SQL indexing, aliases and snapshots. An adapter's emulation is not proof of native support.

Typical sequence

Start only the required Compose profile. Wait for health and index readiness. Create an owned collection, upsert, verify replacement, query with filters, delete, then confirm deletion. Back up and restore before trusting persistence.

Reproducible comparison

uv run --extra embedded python capstones/02_database_bakeoff/solution/project.py
The small local comparison uses SQLite-vec, Chroma and LanceDB. Reject any run with errors or insufficient recall before comparing latency.

Avoid misleading claims

A 500-row winner may lose at larger scale or under concurrent writes. Measure operational requirements separately. Pin server images and clients, bind demo ports to loopback, and delete only course-owned resources. See docs/CHOOSING_A_VECTOR_DATABASE.md.